

For example, consider this *countDownAndUp.py* program whose recursive function counts from any number down to zero, and then back up to the number:

```
Python def countDownAndUp(number):
    ❶ print(number)
    if number == 0:
        # BASE CASE
        ❷ print('Reached the base case.')
        return
    else:
        # RECURSIVE CASE
        ❸ countDownAndUp(number - 1)
        ❹ print(number, 'returning')
        return

    ❺ countDownAndUp(3)
```

Here is the equivalent *countDownAndUp.html* program:

```
JavaScript <script type="text/javascript">
function countDownAndUp(number) {
    ❶ document.write(number + "<br />");
    if (number === 0) {
        // BASE CASE
        ❷ document.write("Reached the base case.<br />");
        return;
    } else {
        // RECURSIVE CASE
        ❸ countDownAndUp(number - 1);
        ❹ document.write(number + " returning<br />");
        return;
    }
}

    ❺ countDownAndUp(3);
</script>
```

When you run this code, the output looks like this:

```
3
2
1
0
Reached the base case.
1 returning
2 returning
3 returning
```

Remember that every time a function is called, a new frame is created and pushed onto the call stack. This frame is where all the local variables and parameters (such as *number*) are stored. So, there is a separate